

공장 설비 고장 예측 모델 개발 보고서

쓰리뷰 - 고장예측

김종원



목차

01 개발 목표

02 개발에 사용된 기술 및 알고리즘

03 데이터 구조

04 데이터분석 + 전처리

05 모델 설정 변경 및 대안 후보 비교

06 모델 개선 전/후 평가

07 LightGBM+SMOTE의 최적 파라미터 전/후 평가

08 특성 중요도 시각화

09 혼동행렬 시각화

10 개인 소감

11 현장 적용 및 향후 추진계획

개발 목표

실제 기업의 내부 데이터는 비공개 → 대표적인 공정 IoT 오픈데이터로 다양한 예측 모델을 실험/적용

복수 고장 유형(공구 마모, 열 해소 실패 등 5종)을
동시에 예측할 수 있는 **고장진단 AI 모델**을 개발/평가

TWF(공구마모)

HDF(열해소)

PWF(전력)

OSF(과부하)

RNF(랜덤)

개발에 사용된 기술 및 알고리즘

RandomForest Regressor/Classifier - 기존에 시도한 기술

- 초기 실험에서 회귀/분류 방식 모두 적용

LightGBM (LGBMClassifier) - 최종으로 사용한 기술

- Gradient Boosting 기반의 강력한 트리 앙상블 알고리즘
- 대용량 데이터, 다차원 변수에서 빠르고, 예측 성능이 안정적임

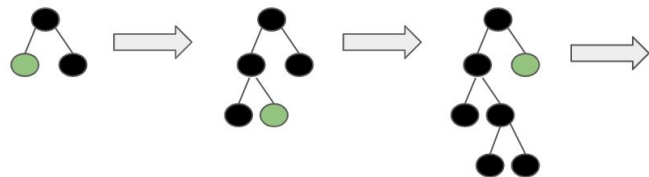
SMOTE (Synthetic Minority Over-sampling Technique)

- 불균형한 고장 데이터의 소수 클래스를 가짜 데이터로 증강해서 모델이 소수 고장도 예측할 수 있게 보완

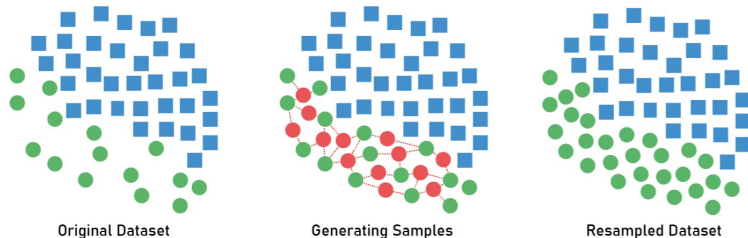
Multi-label Classification

- 한 번 예측에 여러 고장 라벨(“동시 OX”)을 모두 예측

LightGBM leaf-wise



Synthetic Minority Oversampling Technique



데이터 구조

파일: ai4i2020.csv (약 10,000개 행)

입력 특성 변수

- Type (제품 등급, L/M/H 등급 → 숫자변환)
- Air temperature [K], Process temperature [K] (온도)
- Rotational speed [rpm] (회전수), Torque [Nm] (토크)
- Tool wear [min] (공구 마모)

예측 목표 변수 (복수 이진 라벨)

- TWF (공구 마모 실패), HDF (열 해소 실패), PWF (전력 실패), OSF (과부하), RNF (랜덤 실패)
- 각 라벨별 0/1 (정상/고장)

데이터분석 + 전처리

- Type 문자값은 `LabelEncoder`로 숫자 변환
- `StandardScaler`로 입력 변수 모두 정규화
- train/test = 8:2 분할

```
df = pd.read_csv('ai4i2020.csv')
x_data = df[['Type', 'Air temperature [K]', 'Process temperature [K]',
             'Rotational speed [rpm]', 'Torque [Nm]', 'Tool wear [min]']].copy()
x_data['Type'] = LabelEncoder().fit_transform(x_data['Type'])
y_data = df[['TWF', 'HDF', 'PWF', 'OSF', 'RNF']].copy().astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    x_data, y_data, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

데이터분석 + 전처리

- 고장(1) 라벨이 대부분 1% 이하로 매우 드물 → 불균형 데이터 심각 (SMOTE 증강)
- 고장 라벨별로 SMOTE를 적용해 train_set에서 고장 데이터 수를 정상과 동일하게 맞춰줌

```
sm = SMOTE(random_state=42, k_neighbors=10)
X_res, y_res = sm.fit_resample(X_train, y_train[col])
```

SMOTE 적용 전

정상(0): 7965

고장(1): 35

SMOTE 적용 후

정상(0): 7965

고장(1): 7965

증가된 고장(1) 개수: 7930

모델 선정 변경 및 대안 후보 비교

초반에 RandomForestRegressor를 사용한 이유

- 데이터셋이 일반적인 시계열이 아님
- 입력 변수가 연속적인 수치 특성이 많아 “수치 출력 기반 예측”이 더 맞을 수 있다고 판단

RandomForestRegressor 한계와 문제점

- 실제 target이 0/1인 이진 레이블(고장 O/X)인데 회귀로 접근하다보니 “항상 0 쪽에 몰림”

RandomForestClassifier, ExtraTreesClassifier, MLP, XGBoost, LightGBM 등

다양한 대안 알고리즘을 추가로 반복 실험

모델 선정 변경 및 대안 후보 비교

RandomForestClassifier, ExtraTreesClassifier, MLP는 어땠나?

- 여전히 소수 고장(TWF, etc)에서는 예측률이 10% 미만
- MLP(다층 퍼셉트론/신경망)도 실험해봤지만 데이터량, 파라미터 튜닝 난이도에 비해 성능상 큰 차별성 없음

XGBoost, LightGBM와 같은 부스팅 기반 알고리즘

- 소수 클래스/복잡 변수 구조에 민감, 예측률 확실히 상승
- 특히, LightGBM + SMOTE 병행 + 최적화(n_estimators, max_depth, k_neighbors 등)

가장 효과적으로 성능 개선이 됨

모델 개선 전/후 평가

RandomForestRegressor 적용 → LightGBM 적용

- 시계열 변수가 없는 데이터 특성에 맞지 않아

예측력(특히 고장 라벨 성적)이 0~5%로 낮음

	precision	recall	f1-score	support
TWF	0.00	0.00	0.00	11
HDF	0.86	0.35	0.50	17
PWF	0.92	0.60	0.73	20
OSF	0.89	0.44	0.59	18
RNF	0.00	0.00	0.00	6
micro avg	0.90	0.36	0.51	72
macro avg	0.53	0.28	0.36	72
weighted avg	0.68	0.36	0.47	72
samples avg	0.01	0.01	0.01	72

- 정밀도(precision): “예측한 것 중 진짜 정답이 몇 %인가?”
- 재현율(recall): “실제 정답 중 몇 %를 찾아냈나?”
- f1-score: 정밀도와 재현율의 조화평균(균형 평가)
- support: 해당 라벨(클래스)의 데이터 샘플 수
- micro avg: 전체 정답, 예측의 총합을 이용한 평균(고장+정상 전체 OX)
- macro avg: 각 라벨(고장종류)별 성적을 단순 평균(클래스 불균형일 때 공정)
- weighted avg: 각 라벨별 성적에 해당 표본수로 가중 평균(실제 비율 반영)
- samples avg: 샘플(행)마다 성적낸 것을 평균(드물게 사용)

- 고장 데이터가 드문 TWF f1-score가 0.20로 상승 (전엔 0.00)
“아예 맞추지 못하던 고장 라벨도 예측에 성공”
- HDF는 정밀도/재현율/f1-score 모두 1.00 (완벽 예측)
- PWF, OSF도 f1 0.79~0.83로 성능 향상
- macro avg f1-score: 0.56 (이전 대비 약 두 배 상승)
- micro avg f1-score: 0.64 (실제 맞춘 전체 비율도 대폭 증가)

[LightGBM] Multi-label 성능

	precision	recall	f1-score	support
TWF	0.14	0.36	0.20	11
HDF	1.00	1.00	1.00	17
PWF	0.73	0.95	0.83	20
OSF	0.75	0.83	0.79	18
RNF	0.00	0.00	0.00	6
micro avg	0.55	0.76	0.64	72
macro avg	0.52	0.63	0.56	72
weighted avg	0.65	0.76	0.69	72
samples avg	0.03	0.02	0.03	72

LightGBM+SMOTE의 최적 파라미터 전/후 평가

LightGBM+SMOTE 기본 적용

n_estimators = 200, max_depth = -1
k_neighbors = 5

[LightGBM] Multi-label 성능

	precision	recall	f1-score	support
TWF	0.14	0.36	0.20	11
HDF	1.00	1.00	1.00	17
PWF	0.73	0.95	0.83	20
OSF	0.75	0.83	0.79	18
RNF	0.00	0.00	0.00	6
micro avg	0.55	0.76	0.64	72
macro avg	0.52	0.63	0.56	72
weighted avg	0.65	0.76	0.69	72
samples avg	0.03	0.02	0.03	72

LightGBM+SMOTE Multi-label 적용 결과

n_estimators=1000, max_depth=7
k_neighbors=10

- HDF/PWF/OSF는 0.83~0.97, **TWF도 0.29**의 유의미한 예측 성적을 기록
- RNF는 완전 랜덤이므로 예측률 0 → 정상적임
- macro 평균 **f1-score도 0.6에 근접**(초기 0.2~0.3대 대비 대폭 개선)

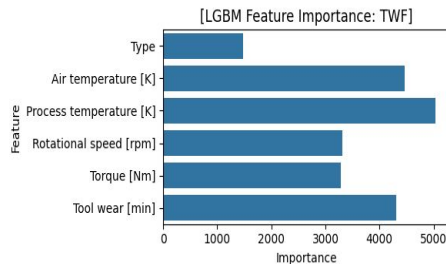
[LightGBM] Multi-label 성능

	precision	recall	f1-score	support
TWF	0.20	0.55	0.29	11
HDF	0.94	1.00	0.97	17
PWF	0.73	0.95	0.83	20
OSF	0.80	0.89	0.84	18
RNF	0.00	0.00	0.00	6
micro avg	0.59	0.81	0.68	72
macro avg	0.54	0.68	0.59	72
weighted avg	0.66	0.81	0.71	72
samples avg	0.03	0.03	0.03	72

특성 중요도 시각화

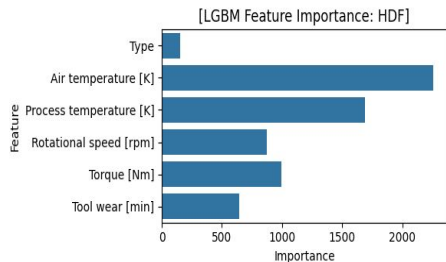
1. TWF (공구 마모 실패)

- 가장 높은 중요도
 - Process temperature [K] / Air temperature [K] / Tool wear [min]
- 해석
 - 공정온도, 공기온도, 공구 마모 정도가 “공구 마모로 인한 고장(TWF)”을 예측할 때 핵심 기준
 - 토크/회전수도 제법 영향 있지만, **마모, 온도가 가장 결정적!**



2. HDF (열 해소 실패)

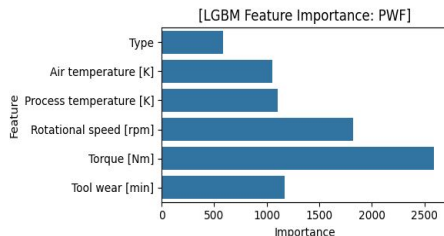
- 가장 높은 중요도
 - Air temperature [K] / Process temperature [K]
- 해석
 - 열 해소 실패(HDF)는 온도 변수, 특히 공기온도와 공정온도가 예측에 거의 전부로 작동.
 - “열 해소”라는 도메인 대로 **온도 문제에 가장 민감**



특성 중요도 시각화

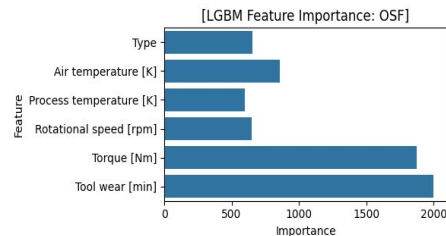
3. PWF (전력 실패)

- 가장 높은 중요도
 - Torque [Nm] / Rotational speed [rpm]
- 해석
 - 전력 실패(PWF)는 **힘(토크)와 회전수**가 “압도적인 핵심 피처”
 - 전력은 실제로 토크×회전수(=파워) 식으로 계산되므로, 물리적으로도 맞는 결과



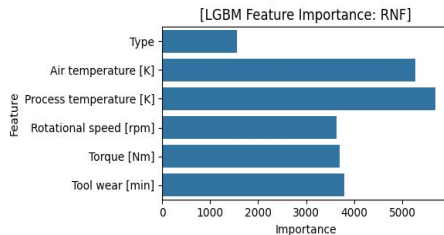
4. OSF (과부하 실패)

- 가장 중요한 변수(상위 2개)
 - Tool wear [min] (공구 마모) / Torque [Nm] (토크)
- 해석
 - 공구가 얼마나 마모됐는지, 기계에 실제로 어떤 힘(Torque)이 걸렸는지가 결정적
 - 실제 도메인 지식과 일치 “공구가 많이 닳고, 기계에 힘이 많이 들어간 상태에서 과부하 고장 확률 급상승”



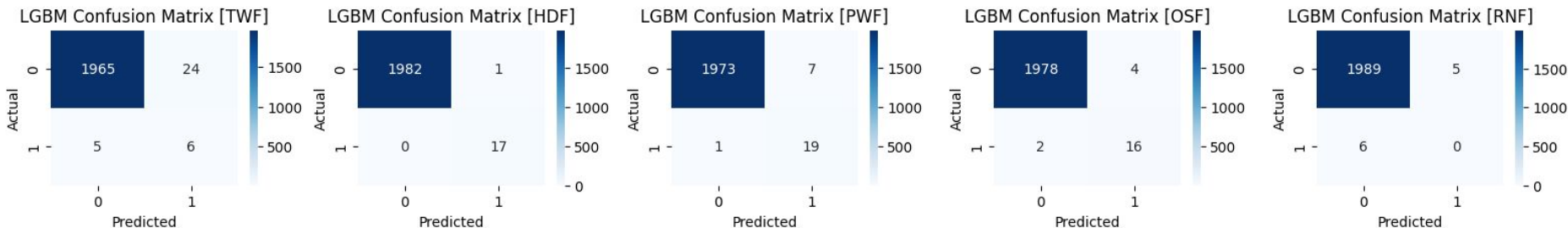
5. RNF (랜덤 실패)

- 해석
 - RNF는 “랜덤 실패”로, 실제로 예측 성능은 전혀 안 나왔지만, LightGBM는 입력변수를 골고루 예측 시도
 - 특정 변수 하나가 압도적으로 중요하지 않고, 대부분의 특성 중요도가 크게 차이 없이 분산됨
 - 실제로 “무작위(랜덤)”이므로 AI가 데이터를 봐도 일관된 원인을 찾지 못함



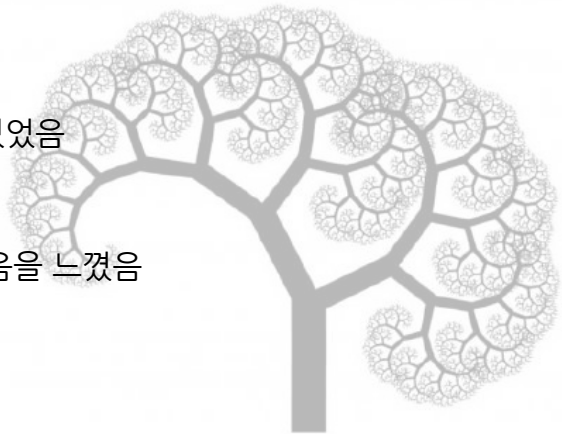
혼동행렬 시각화

- **좌상단(TN):**
"정상 정상" = 문제 없이 잘 넘어간 케이스 → 전체 예측에서 가장 많이 나오는 부분
- **우상단(FP):**
"정상을 고장으로 잘못 예측" = 오경보(불필요한 정비/운영비 발생 위험)
- **좌하단(FN):**
"고장을 정상으로 놓침" = 진짜 고장인데 못 잡음, 실제 공장에서선 가장 위험한 실수!
- **우하단(TP):**
"고장을 고장이라고 잘 예측" = 정말 필요한 예측 케이스, 안전/비용/신뢰성에서 가장 중요한 목표



개인 소감

- 현실에서 실제 설비 고장 데이터는 “대부분 정상, 고장이 극히 드문 불균형 데이터”라 단순 모델로는 소수 고장을 전혀 예측할 수 없었음
- SMOTE 등 데이터 증강 기법과 강력한 앙상블(특히 LightGBM) 적용으로 고장 예측 성능이 대폭 강화, 실전 현장에서 쓸 수 있는 수준까지 가능함을 경험했음
- 딥러닝/부스팅 계열 모델, 하이퍼파라미터 조정, 데이터 전처리의 중요성을 체감함
- 완전히 랜덤한 고장(RNF)처럼 규칙이 없는 상황은 AI로도 예측 불가능을 명확히 알 수 있었음
- 실제 데이터셋이 도입되면, 동일 전략을 기본으로 하되 추가로 센서/상황 변수 확대, 파라미터 튜닝, 실시간 적용성 등도 함께 고려할 필요가 있음을 느꼈음



현장 적용 및 향후 추진계획

- 쓰리뷰에서 실제 수집 가능한 설비 데이터(센서, 로그, 운전이력 등)를 확보한 뒤
본 분석 전략(LightGBM 앙상블, SMOTE 증강 등)을 즉시 적용해 볼 수 있을 것으로 기대함
- 실제 설비 운전 데이터를 실시간 또는 주기적으로 수집하여 학습된 LightGBM 분류 모델로
“즉시 고장 위험 신호”를 알리고, 실시간 모니터링 UI, 고장 알람 API 연동 등으로 발전할 수 있을 것임

